

Where LLM Layout Generation Fails: An Open-Source Reproduction of LLM-Driven Template-and-Grid Layout and an Interface Layer That Makes It Self-Healing

Jongkyoung, Han

Independent researcher

<https://github.com/Toragonite/laygo2-llm-agent>

Abstract—You et al. (MLCAD 2024 WIP) proposed letting a large language model (LLM) write Python layout generators for the template-and-grid framework LAYGO2 from natural-language instructions, with the designer inspecting the result and asking for modifications. We reproduce this flow on a fully open-source stack (public laygo2, SkyWater SKY130, Magic, netgen) with a scripted DRC/LVS judge, a fixed protocol for the human’s instructions, and a failure taxonomy. On eight transistor-level cells (2–11 transistors, including the paper’s 2-to-1 MUX, ratioed TSPC flip-flop and strong-arm latch), five attempts each, GPT-4o passes 0 of 40 attempts on the six context turns alone and 21 of 40 after up to eight human instructions. Labelling every failing round shows that most failures are not circuit mistakes but friction at the LLM–framework interface: misused return values, wires on occupied tracks, and a routing primitive that silently fills a rectangle between misaligned points. We add a thin, net-aware layer, `laygo2_safe`, that keeps laygo2’s architecture and adds validated inputs, an occupancy table, a `connect()` primitive with track search, and netlist-aware checks that fail at the offending line. The same tasks then pass 40 of 40 (13 of 40 on the first turns); fix rounds drop from 225 to 41 and interface-type failure labels from 134/127/85 to 7/5/0. Because every remaining failure is a precise tool message, feeding it back verbatim replaces the human: 40 of 40 pass with no human in the loop (24 of 40 on the first turns, 56 automatic rounds, about \$2.4). Layout area matches the human-written references (ratio 0.98); the automatic router uses more metal2 on the larger cells, which we measure and leave as future work. All logs, prompts and protocols are released.

Index Terms—layout generation, large language models, LAYGO2, SKY130, design automation, benchmark

I. INTRODUCTION

Custom cell layout—logic standard cells and high-speed mixed-signal primitives—is still largely manual. Generator-based frameworks such as BAG and LAYGO/LAYGO2 [1], [2], [3] let designers describe a layout as Python code over process-portable templates and abstract grids, which raises productivity but moves the burden to writing that code. You et al. [4] asked whether an LLM can write the generator from natural language, with the designer steering interactively, and reported thirteen DRC/LVS-clean designs, from NAND gates to a 32 Gb/s serializer, using ChatGPT-4.

Their report leaves questions open for anyone who wants to build on it: the prompt logs are no longer available, the

process node and tools are not stated, each design is reported once (no success rate or variance), and the amount of human help is summarized only as a prompt count. This work makes the setting reproducible and measurable, and then uses the measurement to find and fix the bottleneck. The improvement followed two hypotheses in a fixed order: that the library surface, not the model, is the bottleneck and should be repaired first; and that once every failure is a precise tool message, feeding it back can replace the designer. Our contributions are:

- 1) an open-source reproduction: public laygo2 (pinned), SKY130, Magic DRC and netgen LVS wrapped in a one-command judge that was first validated with human-written reference layouts, including three verification pitfalls that would otherwise produce false passes or false failures (§III-B);
- 2) a measurement protocol for interactive LLM layout generation: four specificity levels for the human’s instructions, a five-type failure taxonomy applied to every failing round, a cap on instructions, and contamination rules (§III-D);
- 3) a baseline on eight cells $\times$ five attempts (§IV) and a failure analysis showing that interface friction, not circuit knowledge, dominates (§V);
- 4) `laygo2_safe`, a thin layer over laygo2 designed from those labels: one return type per function, grid-aware points, an occupancy table calibrated on the reference layouts, a `connect()` primitive with a fixed search order, and netlist-aware checks that fail at the offending line (§VI); it keeps laygo2’s architecture and is validated by rewriting all eight reference layouts on it;
- 5) a test of the second hypothesis, that precise tool messages make the human unnecessary: measurements under human instructions and under purely automatic feedback (§VII), an analysis of how the automatic loop fails and negative results for two remedies, a problem history and a script diff (§VII-B), and quality metrics beyond pass/fail (§VIII).

TABLE I
SOFTWARE STACK.

component	version
laygo2 (public)	commit <code>cc6276a</code> ; SKY130 workspace <code>e7bc4fd</code>
process	SKY130A, open_pdks <code>6d4d117</code>
DRC / extraction	Magic 8.3.460 (from source), deck <code>drc(full)</code>
LVS	netgen 1.5.133, SKY130 setup
LLM (reproduction)	<code>gpt-4o-2024-05-13</code>
LLM (improvement)	<code>gpt-4o-2024-08-06</code> , prompt caching
API settings	provider-default temperature, 4096 output tokens, nothing stored

II. BACKGROUND

Template-and-grid layout. In LAYGO2 a *template* is a pre-drawn, DRC-clean block (a transistor with a given number of fingers, a via), and a *grid* maps integer coordinates to physical positions whose pitches absorb the process’s width and spacing rules. A generator places template instances on a placement grid and draws wires on routing grids by integer track indices; the same code targets another process by swapping the technology file. The abstraction suits an LLM because coordinates are small integers and most design rules are absorbed by templates and grids.

The paper’s flow. The LLM first converts a SPICE netlist into per-transistor dictionaries (`NO = {"D": "O", "G": "A", "S": "INT"}`); then, over several turns in which the designer explains placement rules, alignment strategy, routing commands and track-based routing strategy, it emits placement and routing code. The designer inspects the layout and issues modification instructions until DRC and LVS pass (Figs. 2–4 of [4]). Reported prompt counts range from 3+3 (NAND) to 9+36 (strong-arm latch). The authors rate interactive placement and routing as good, and autonomous optimization as normal (placement) to bad (routing).

III. EXPERIMENTAL SETUP

A. Stack

Table I lists the components. Later laygo2 versions are incompatible with the SKY130 workspace, so the January 2024 commit is pinned. The public release lacks the `netmap` module used by the paper’s authors; since the netlist-to-dictionary step is performed by the LLM, it is not needed.

B. Judge and Its Pitfalls

`flow/check_cell.py` runs a generator from the workspace root, sources the exported Magic script in a fresh Magic process (one process per cell), runs DRC, extracts a SPICE netlist, runs LVS against a reference netlist and prints one JSON line (`gen_ok`, `drc_errors`, `lvs_match`, `area`, `stage times`, `raw error text`). Before any LLM was involved, human-written reference generators for all eight cells had to pass it. Three pitfalls were found and are handled in the judge:

- **LVS false passes.** netgen 1.5.133 prints “Circuits match uniquely” when a transistor width is wrong (it adds “Property errors were found” elsewhere) and when two top-level pins are swapped (it only marks `**Mismatch**`

in the pin table). The LVS wrapper passes only when all three conditions hold; negative controls (wrong width, swapped pins, wrong topology) are rejected for every cell.

- **DRC counts.** On the hierarchical cell, `drc listall count` includes violations of vias and template pieces checked in isolation (17 on a clean inverter). DRC is run with the full deck on a flattened copy of the top cell.
- **Hierarchical extraction.** Magic 8.3.460 silently drops a sub-cell whose only direct contents are via arrays from hierarchical extraction, giving DRC-clean layouts that fail LVS. The gate-level references work around it; the cells used here are flat.

Reference netlists use the transistor sizes the templates actually draw (`nfet_01v8_lvt` $W = 0.5\ \mu\text{m}$ and `pfet_01v8` $W = 1.0\ \mu\text{m}$ per finger, $L = 0.15\ \mu\text{m}$, two fingers), not the $W = 1.2/2.4\ \mu\text{m}$ of the workspace’s example netlists, which the layouts never draw.

C. Tasks

Six cells come from the workspace examples, patched to a single size and to our output paths: INV, NAND2, NOR2, TINV, NAND3 and a 2-to-1 MUX (2, 4, 4, 4, 6 and 10 transistors). Two come from Fig. 9 of [4] and were drawn for this work: a ratioed TSPC flip-flop (7) and a strong-arm latch (11); both use two fingers per device and are connectivity references only, neither sized nor simulated. The LLM receives only the cell name and the reference netlist with comments stripped and device names renamed `XM1`, `XM2`, . . . in file order, so that no layout hint leaks through instance names; the same file is the LVS reference. Each cell is attempted five times.

D. Protocol for the Interactive Loop

The context reconstructs the paper’s Figs. 2–4 as six user turns (netlist $\rightarrow$ dictionaries; placement rules and alignment strategy; placement code; routing commands; pin access and track strategy; routing and the complete script). No example generator is shown. The modification loop is governed by Table II. Instructions were written by four instances of a coding agent (Claude) acting as the designer, two cells each; this differs from the paper’s human designer and introduces instructor variance, which we report.

IV. REPRODUCTION: BASELINE RESULTS

Fig. 1 (red) and Table III give the baseline. No attempt passed on the six context turns alone: 34 of 40 crashed while running (undefined template variables, misread return values, placement helpers applied to pins) and 6 failed LVS. After up to eight instructions, 21 of 40 passed; all cells with ten or more transistors and both high-speed circuits failed every attempt. Total tokens 2.69 M in / 0.36 M out, about \$19. The NAND2 vs NOR2 gap (1/5 vs 5/5 on equal-sized cells) is partly instructor variance: the NOR instructor explained template geometry when the model stalled, the NAND instructor did not. We keep both as they are.

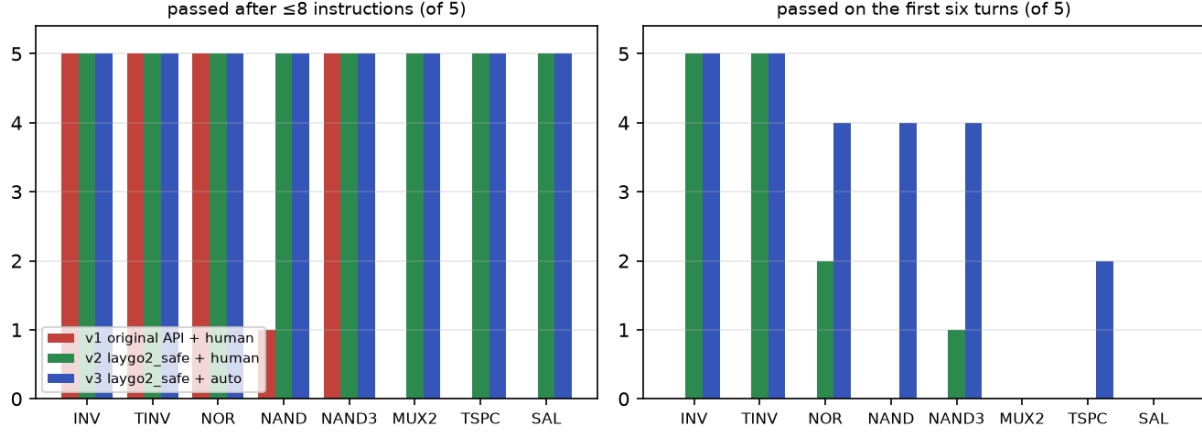


Fig. 1. Passed attempts per cell (left) and attempts that passed on the first six turns (right), for the three conditions.

TABLE II
MEASUREMENT PROTOCOL.

instruction levels	L1 tool output only; L2 diagnosis; L3 what to change and which API/rule; L4 coordinates, column numbers, routing plans. Baseline allows $\leq$ L3.
failure types	A API misuse; B circuit-reading error (ties, memberships, omissions); C routing conflict/short; D geometry misunderstanding; E spacing. Multiple labels per failing round.
budget	$\leq$ 8 instructions per attempt; one instruction = one message.
contamination	instructors never open reference generators or layouts; they see the task netlist, the LLM’s code and conversation, the judge’s outputs and rendered layouts.
record	model, temperature, git hash of prompts and tools, attempt, date, instructor, tokens, per-round labels.

TABLE III

BASLINE (V1): ORIGINAL LAYGO2 API, GPT-4o-2024-05-13, HUMAN-STYLE INSTRUCTIONS $\leq$ L3, FIVE ATTEMPTS PER CELL.

cell	passed	mean instr. (passed)	paper P/R
INV	5/5	2.2	–
TINV	5/5	2.4	–
NOR2	5/5	4.6	3 / 2
NAND2	1/5	7.0	3 / 3
NAND3	5/5	4.0	–
2-to-1 MUX	0/5	–	8 / 15
TSPC FF	0/5	–	3 / 11
strong-arm latch	0/5	–	9 / 36

V. FAILURE ANALYSIS

Every failing round carries one or more labels (Fig. 2). Three interface-type labels dominate the baseline: A (134), C (127) and D (85); B (79) and E (15) follow. The recurring mechanisms were:

- **A.** `route()` returns a wire or a list depending on the number of objects; `route_via_track()` returns a list whose last element is the track wire; placement-grid helpers were applied to pins; coordinates taken on one grid were used on another; a two-dimensional placement

list with rows of unequal length raises a numpy error (6 of 10 first rounds on the larger cells, and the model regressed to it after fixing it).

- **C.** The gate and drain pins of a two-finger template share a column, so the output wire on the drain column shorts the input on the gate column (every inverter attempt); tracks chosen as “pin $\pm$ 1” land on a neighbour’s tied source; a horizontal track on the source row crosses tied sources.
- **D.** `route()` between two points that differ in both row and column fills the whole rectangle between them, shorting everything underneath (Fig. 3b).
- **B.** `tie='S'` applied to every transistor regardless of its own dictionary; multi-pin nets built as one nfet drain plus one pfet drain; a transistor dropped from the list when re-ordering.

The labels say where to intervene: A, C and D are properties of the tool surface the LLM programs against, not of its circuit knowledge, and B is largely a translation slip from a correct dictionary to code. The netlist-to-dictionary turn was correct in all 40 attempts.

VI. LAYGO2_SAFE: A THIN, NET-AWARE LAYER

The improvement phase tested two hypotheses, in a fixed order. First, that the bottleneck is the library surface, so the interface should be repaired before anything is done on the LLM side; the labels of §V motivate this, since A, C and D are properties of the tool. Second, that once the library reports every failure as a precise, actionable message, a loop that feeds that message back can replace the designer. This section describes the layer built for the first hypothesis; §VII-B tests the second.

The layer keeps laygo2 as it is: templates, grids, the design database and the Magic exporter are used unchanged, the pinned submodule is not modified, and the layer (about 850 lines of Python in the repository, from `laygo2_safe import Cell`) sits between the generator script and laygo2. The design goal was not a new layout engine but a surface on which the

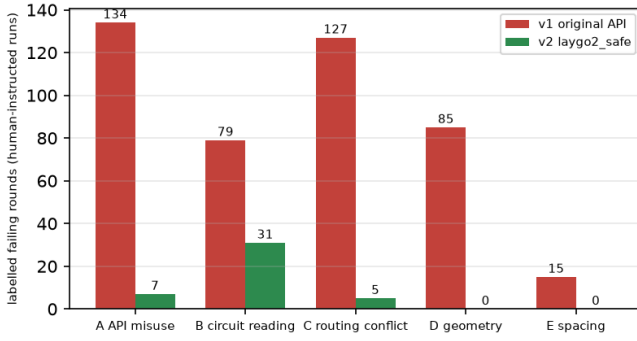


Fig. 2. Failure labels over all failing rounds, baseline (v1) versus the interface layer (v2).

baseline’s failure modes cannot be expressed, or are rejected with a message that names the fix.

A. From Failure Labels to Requirements

Table IV maps each baseline mechanism of §V to the rule the layer enforces and to the code that enforces it. Three decisions shaped everything else. (i) *One kind of return value per function*: `connect()` always returns one wire, `rails()` always returns two, a point always carries the grid it was taken on and is refused on another grid. (ii) *The cell knows the netlist*: the reference netlist is given to the `Cell` constructor, so the layer can check ties, finger counts and nets at the line that states them instead of after extraction. (iii) *The cell knows what is occupied*: every pin, wire and via is recorded as physical rectangles with a net, so a short or a spacing violation is detected when the wire is requested, by the layer, instead of by Magic or netgen minutes later.

B. The API

Fig. 4 is the complete NAND2 generator on the layer, 23 lines against 123 for the workspace’s original generator. The script contains no coordinates, no track indices, no via flags and no grid conversions; the only geometric decisions left to the author are which grid a net uses (`r12` inside a row, `r23` between the rows) and, optionally, which end of a spanning pin (`'left'`/`'right'`). This is also the entire vocabulary the LLM is taught in the rewritten context: `Cell`, `nmos/pmos`, `place_rows`, `rails`, `connect`, `port`, `check`, `export`.

C. Occupancy Table and Its Calibration

laygo2 exposes pins as centrelines on abstract grids, not as the metal the template actually draws, so the layer cannot ask the framework how much space a pin takes. The footprints were therefore measured on the flattened CIF of the human-written reference cells (Magic crashes when asked to write a template cell’s CIF on its own) and are stored in grid units of 5 nm (Table V). Each placed instance registers: its drain and source contact strips, the `li1` bar along the source row, the gate strap, its rail segment (on the rail net), and, for a tied terminal, the contact strips and the `li1` straps down to the rail, all on the

TABLE IV
BASELINE FAILURE MECHANISMS AND THE LAYER’S RESPONSE.

label	baseline mechanism	rule in the layer (implementation)
A	<code>route()</code> returns a wire or a list; <code>route_via_track()</code> returns a list whose last element is the track	every function returns one kind of object; <code>connect()</code> returns the main wire that <code>port()</code> needs
A	coordinates of one grid used on another; placement helpers applied to pins	<code>Point</code> remembers its grid and is refused on another; pins are named, never indexed
A	ragged 2-D placement list raises a numpy error; a created device left unplaced	<code>place_rows(nfets, pfets)</code> accepts rows of different length and refuses an unplaced device by name
D	<code>route()</code> between points on different rows and columns fills the rectangle between them	<code>wire()</code> refuses two points not on one track; paths are only drawn by <code>connect()</code>
C	output wire on the drain column shorts the gate pin in the same column; “pin ± 1” tracks land on a neighbour’s tied source	occupancy table of physical rectangles per layer with a net; any wire that touches another net or is within the layer’s spacing is rejected and the obstacle is named
C, E	the model has to choose a free track	<code>connect()</code> searches tracks and paths in a fixed order (§VI-D) and takes the first legal one
B	<code>tie='S'</code> applied regardless of the dictionary; wrong net on a pin; pin dropped	<code>nmos(..., ref='XM1')</code> refuses a tie or finger count that contradicts the netlist; <code>connect()</code> refuses a pin whose netlist net differs; <code>check()</code> lists unconnected pins, split nets and missing ports
–	two port labels at one centre; Magic keeps one	<code>port()</code> places the label on a sub-segment of the wire that no other pin centre shares

TABLE V
PIN FOOTPRINTS AND SPACING USED BY THE OCCUPANCY TABLE (1 UNIT = 5 NM; MEASURED ON THE REFERENCE CELLS).

item	extent	layer
drain contact strip	±34 across; 105 toward the source row	li1
source contact strips	±24 across; 113 toward the drain row	li1
source bar along the pin row	±45 (0.225 μ m) past both contacts	li1
gate strap	±40 laterally	li1
rail segment	±26 vertically	metal1
tied-terminal straps	±26 across, pin row to rail	li1
via pads	±17 (li1), ±26 (metal1, metal2)	–
minimum spacing	34 (li1), 28 (metal1), 28 (metal2)	–

rail net. Wires and vias added by `connect()` are registered with their net as they are committed. A candidate wire is legal when none of its rectangles overlaps a rectangle of another net on the same layer and none is closer than the layer’s minimum spacing; rectangles of the same net may touch.

The calibration was done against the reference layouts: the layer had to accept every wire of the eight human-written cells and reject each negative control. The first model over-approximated the tied-source strips up to the drain row and rejected the reference NAND’s output wire as touching VDD; the measured strip lengths (105 and 113 units) resolved it. The

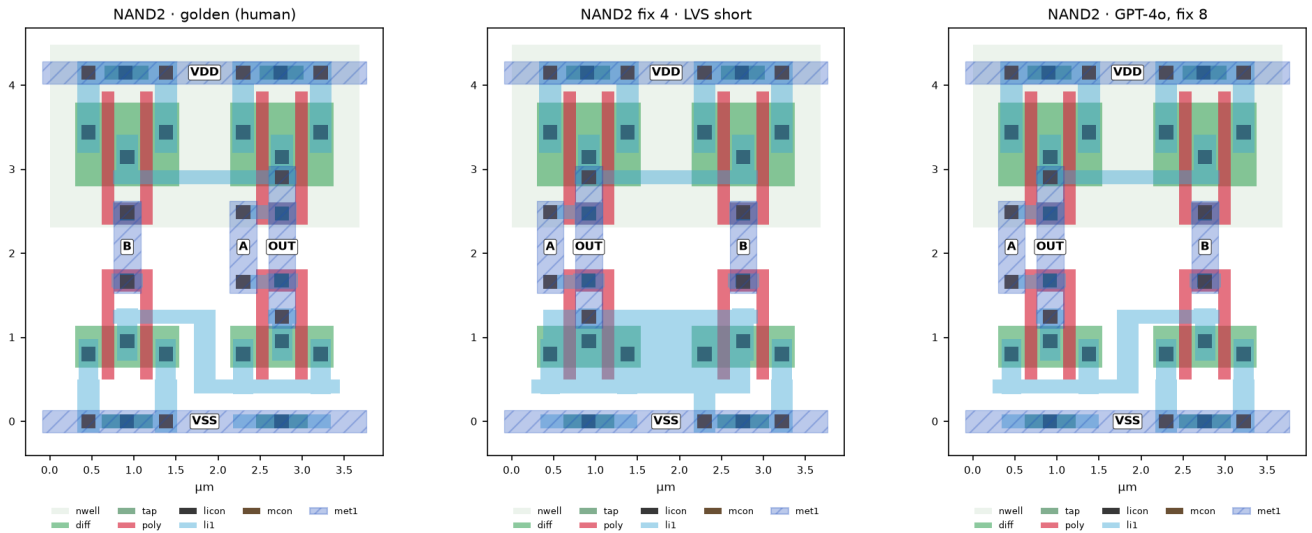


Fig. 3. NAND2. (a) Human reference. (b) The baseline model after four instructions: DRC-clean but shorted by one `route()` call between points on different rows, which fills a rectangle across the nfet row (light blue). (c) The model's passing NAND2 after eight instructions, a mirror image of the reference.

```
from laygo2_safe import Cell

c = Cell("nand_2x", netlist="nand.spice")
n1 = c.nmos("MN1", nf=2, ref="XM1") # OUT-A-net1
n0 = c.nmos("MN0", nf=2, tie="S", ref="XM2") # net1-B-VSS
p0 = c.pmos("MP0", nf=2, tie="S", ref="XM3") # OUT-B-VDD
p1 = c.pmos("MP1", nf=2, tie="S", ref="XM4") # OUT-A-VDD
c.place_rows([n0, n1], [p0, p1])

c.connect("OUT", [(p0, "D", "right"), (p1, "D")], "r12")
w_out = c.connect("OUT", [(n1, "D", "right"),
                          (p1, "D", "right")], "r23")
w_a = c.connect("A", [(n1, "G"), (p1, "G")], "r23")
w_b = c.connect("B", [(n0, "G"), (p0, "G")], "r23")
c.connect("net1", [(n0, "D"), (n1, "S")], "r12")
w_vss = c.connect("VSS", [(n0, "RAIL"),
                          (n1, "RAIL", "right")], "r12")
w_vdd = c.connect("VDD", [(p0, "RAIL"),
                          (p1, "RAIL", "right")], "r12")

for name, g, w in [("A", "r23", w_a), ("B", "r23", w_b),
                  ("OUT", "r23", w_out), ("VSS", "r12", w_vss),
                  ("VDD", "r12", w_vdd)]:
    c.port(name, g, w)
problems = c.check()
assert not problems, problems
c.export()
```

Fig. 4. The reference NAND2 generator on `laygo2_safe` (DRC0, LVS match; netlist path shortened). The original generator for the same cell needs 123 lines, including per-net grid conversions, track indices and via flags.

source bar's lateral extent ($0.225\mu\text{m}$ past the contacts) was found from a NAND3 li1 spacing violation that the first model had let through, and the via pad size from metal1 violations next to pins. Each such correction is a case where the human-written layouts served as ground truth for the layer, in the same way they had served as ground truth for the judge.

D. `connect()`: Search Order

`connect(net, pins, grid)` first validates: every pin's netlist net must equal `net`; a pin already assigned to another net is refused; a RAIL pin can only carry its own rail

net. Pins that span several columns (S, RAIL) and are given without an end contribute both ends as candidates, and the Cartesian product of end choices is tried in order. For one choice of points the router tries, in this order, and commits the first legal candidate:

- 1) a straight wire, if all points share a row or a column (vias where the pin layer differs from the wire layer);
- 2) a shared track *between* the pins (shortest stubs), vertical family first unless the points share a row;
- 3) for two pins, L- and Z-shaped three-segment paths: the middle run on one of the two pin rows (an L), then on rows between the pins, then on rows up to six tracks outside; the same mirrored with a column;
- 4) shared tracks outside the pins, up to six tracks away on either side;
- 5) for two pins on `r23`, an escape on metal2: a via up at the first pin, metal1 to a free row of grid `r34` (which shares `r23`'s columns), metal2 across, metal1 down and, if the descent column differs, a short li1 stub to the second pin;
- 6) for more than two pins, the first pin is joined to each other pin by a recursive two-pin call.

A call on `r12` with no legal li1 path (typically the $0.17\mu\text{m}$ li1 rule next to a source bar) is retried on `r23`, which makes the metal2 escape available. If everything fails, the error lists the first six candidates with the rectangle that blocked each, and ends with "try another grid, a different end of the pin, or move the instances apart". Vias are added only at ends that are pins; an earlier version drew a via at the free end of a single-pin stub and produced an isolated li1 pad (rule li.6).

E. Netlist-Aware Checks and Messages

The checks run at three moments, each as early as the information allows:

- **At device creation.** With `ref='XM1'`, `nmos()` compares kind, multiplier and tie with the netlist. Example message: `MN0 (XM1): its S is on VSS in the netlist; use tie='S'; or XM3 is a pmos in the netlist; use c.pmos() for MN2.`
- **At connection.** `MN1.G (XM1.G)` is on net 'A' in the netlist, not 'B'; `MP0.D` is already on net 'OUT'; it cannot also be on 'net1' (check the netlist).
- **At check(),** before export: unconnected pins (`MP2.D` is not connected (`netlist: OUTM`)), wrong nets, wrong ties, ports missing or not in the netlist, and nets left in separate pieces, found by a union-find over the occupancy rectangles of the net (same-layer overlap or a via joins two pieces): net `OUTP` is in 2 separate pieces: `MN3.G`, `MP0.G` | `MN4.D`, `MP1.D` (connect them with a shared pin).

Every message names the instance, the terminal, the netlist fact and the action. This is deliberate: in the v2 runs, 19 of the 22 human instructions were these messages relayed with one sentence of explanation, which is what made the automatic loop of §VII-B plausible.

F. Development: Validation and Defects Found

Validation was twofold. First, a suite of 23 guardrail tests, each asserting that a specific misuse raises a specific message (odd `nf`, a `pmos` in the `nfet` row, a single instance instead of a row, grid mixing, a non-collinear wire, a track through a pin, a manual wire touching another net, a track too close to a drain, a track on the tied-source row, a missing device in `place_rows`, a duplicate port, ...) and that the netlist checks report an unconnected pin, a wrong tie and a split net, and nothing when the cell is complete. Second, all eight reference cells were rewritten on the layer using only `connect()` (18–45 lines each) and pass DRC0 / LVS match; the MUX becomes 16% smaller than the human reference because no spacer cells are needed. Table VI lists the defects found in the layer itself during this development and during the v2 measurement; the four found by the v2 instructors were fixed after the v2 runs and before v3.

G. Prompt Changes Between v1 and v2

The six-turn context was rewritten for the new API with the same structure (netlist → dictionaries; placement rules; placement code; connection commands; connection strategy; connect and finish), the same length, and still without any example generator. Beyond replacing the command vocabulary, five wording changes were made, listed here so that readers can separate them from the library effect: (1) the tie rule, “a device whose source is on VSS uses `tie='S'`”, became “decide `tie` for each device separately from its own dictionary, never for all devices at once ... most cells mix tied and untied devices”; (2) the placement turn asks for a comment on each device line with its D/G/S values and the tie derived from them; (3) the dictionary turn adds a `"ref"` key with the netlist device

TABLE VI
DEFECTS OF THE LAYER FOUND DURING DEVELOPMENT (D) AND BY THE V2 INSTRUCTORS (I), WITH THEIR FIXES.

	symptom	cause → fix
D	reference NAND output wire refused as touching VDD	tied-source strips modelled up to the drain row → measured lengths (Table V)
D	NAND3 li1 spacing violation accepted	source bar too short → $\pm 0.225 \mu\text{m}$ past the contacts
D	long detours with far tracks	outward search first → between-pin tracks, then L-paths, then outward
D	a net on a free row blocked a later net (strong-arm latch)	no third layer → metal2 escape via <code>r34</code>
D	port label lost when two pin centres coincide; LVS pin missing	label at the pin centre → label on a sub-segment at least one track long with no other centre
D	isolated li1 pad (li.6) at a stub end; far-end via missing on multi-pin nets	via at a non-pin end / end flag lost in the pairwise join → vias only at pin ends, per pin
D	reference MUX refused	rule “every later <code>connect()</code> of a net shares a pin” too strict → replaced by the split-net check
D	INV: a rail given as one pin drew a zero-length stub (DRC)	single spanning pin treated as a point → the wire runs along the whole pin; the v2 run in progress was discarded and restarted
I	vertical metal1 port written on li1	zero-length fallback box in <code>port()</code> → layer taken from the wire
I	a rail covering only some devices not reported	no coverage check → <code>rails()</code> draws both rails over all devices
I	second <code>connect()</code> of a net without a shared pin accepted	no connectivity check → split-net check in <code>check()</code>
I	<code>KeyError</code> instead of a message for an unplaced device (four rounds lost)	missing validation → <code>place_rows()</code> names the unplaced devices

name; (4) the connection turns `add rails()`, the instruction to list all pins of a multi-pin net in one `connect()` call, and an order (rails, series nodes, outputs, inputs); (5) the required skeleton ends with a loop that prints `check()`'s findings as `CHECK:` lines, which the judge and the automatic loop read. Everything about `mn` arrays, `via_tag`, track indices and pin indexing was removed.

VII. RESULTS WITH THE LAYER

A. Human-Style Instructions (v2)

Same eight cells, five attempts, `gpt-4o-2024-08-06`, instructions $\leq$ L3 by four coding-agent instructors. Thirteen of 40 attempts passed on the six turns alone (INV and TINV 5/5), and 40 of 40 passed after instructions, with 41 fix rounds in total (v1: 225). Labels fell to A7, B31, C5, D0, E0 (Fig. 2). Nineteen of the 22 instructions were L2 (relaying the tool's message with a one-line explanation); none was L4. The remaining B failures had three forms: a tie set contrary to the device's own dictionary (6 of 10 first rounds on the high-speed cells), multi-pin nets built as a single drain pair (parallel devices and fan-out gates omitted), and one `pfet` dropped from the placement list when re-ordering. Each was repaired by a single instruction quoting the library's message. Tokens 0.76 M in (0.58 M cached) / 0.08 M out, about \$2. A first formal run was discarded: a library bug drew a rail given as a single pin

TABLE VII

COMPOSITION OF THE AUTOMATIC FEEDBACK MESSAGE. EACH BLOCK IS INCLUDED ONLY WHEN ITS CONDITION HOLDS; THE LAST LINE IS ALWAYS PRESENT.

condition	content
script crashed	“The script failed while running.” and the last four non-empty lines of the traceback (for a <code>SafeError</code> , its message)
script ran	the <code>CHECK:</code> lines printed by <code>check()</code> (at most 12)
DRC > 0	the count and the first ten lines of the DRC report: rule name and the violation box in μm
LVS failed	the result class (mismatch / pin mismatch / property errors), netgen’s lines with net and device counts, <code>**Mismatch**</code> , “no matching” and “Property error” (at most 14), and the extracted top-level subcircuit of the layout (at most 1500 characters)
always	“Fix the problems and output the complete corrected script.”

as a zero-length stub and failed DRC on INV; the library was fixed and all 40 attempts were re-run so that the tool version is uniform.

B. Automatic Feedback Only (v3)

The second hypothesis was tested by removing the human: after the six context turns, a script (`llm/selfheal.py`) runs the judge, builds a message from the tool output alone, sends it, extracts the new script and repeats, for at most eight rounds. Table VII gives the composition of the message. It contains no diagnosis and no hint, i.e. it is an L1 instruction in the protocol’s terms, so the comparison with v2 (mostly L2) isolates what the human added. The whole conversation is resent every round; prompt caching makes this cheap (0.75 M of the 0.95 M input tokens were cached).

The library had meanwhile received the four fixes marked I in Table VI, so v3 is not the same tool version as v2. Table VIII and Fig. 5 summarize. All 40 attempts pass; 24 pass on the six turns alone (v2: 13), which reflects the later library version and the slightly different prompts rather than the feedback, since no feedback has been sent at that point. Small cells now pass on the first turns (NAND2, NAND3, NOR2 four of five). The larger cells need two to three times more automatic rounds than human rounds (MUX 3.4 vs 1.0, strong-arm latch 5.2 vs 2.8) but all pass within the budget, at zero human time. Fig. 6 shows a strong-arm latch produced on the layer.

How the automatic loop fails. The extra rounds on the large cells come from one pattern: the loop oscillates between a crash and an LVS mismatch. The first strong-arm attempt went `crash` $\rightarrow$ `LVS` $\rightarrow$ `crash` $\rightarrow$ `LVS` $\rightarrow$ `crash` $\rightarrow$ `LVS` $\rightarrow$ `pass`. Round 1: MN0 (XM1): its S is on VSS in the netlist; use tie=’S’. Round 2: the tie is fixed, but `check()` reports MP2.D is not connected and net OUTP is in 2 separate pieces, and netgen counts 14 nets against 10. Round 3: while adding the missing connections the model regenerated the device block and set a wrong tie on another device (MP2 (XM8): its S is

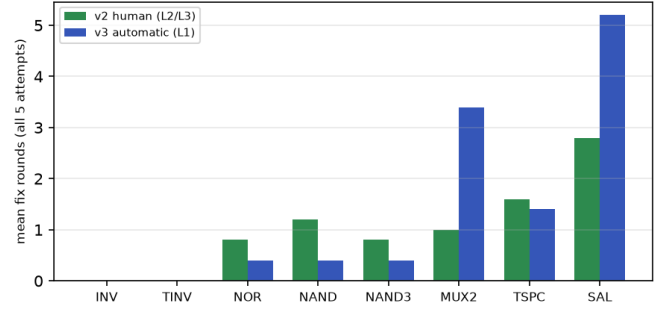


Fig. 5. Mean fix rounds per cell, human instructions (v2) versus automatic feedback (v3), all five attempts.

on VDD . . .), a crash again. Each reply is a complete new script, and a fix in one part of it undoes an earlier fix in another. The human instructors did not show this pattern because their one-sentence explanation usually named both problems at once.

Two negative results. The obvious remedy is memory: a second feedback variant appends, after the current findings, a list of every problem seen in earlier rounds marked *still open*, *fixed—keep it that way*, or *back again (it was fixed before)*, and asks the model to fix the open ones “while keeping everything that already works”. On paired copies of the ten MUX and strong-arm first-round states (43 rounds, 10 of 10 under the plain feedback), this gave 42 rounds and one attempt out of budget. A third variant shows the model what it changed: whenever the failure state flips between the last two scripts (crash to LVS mismatch or back), the unified diff between them is appended with the sentence “do not undo what the previous script already had right”. On the same ten states it gave 41 rounds and again one attempt out of budget; the diff was present in 18 of the 41 rounds, and the kinds of crash were unchanged (wrong tie, pin on a wrong net, no free path: 4/3/2 under plain feedback, 5/3/3 with the diff). Neither variant is used. Tracing the strong-arm attempts shows what the oscillation is: a cycle between two incomplete partial solutions of one coupled problem, the ties of the cross-coupled pmos pair, the inclusion of the precharge drains in the output nets, and a five-pin net for which the router finds no path, and each reply bundles several edits, so a fix in one part of the script undoes another. Neither a history nor a diff made the model unbundle them. The remaining candidates act on the loop rather than on the message: freezing the device and placement block once it passes the library’s checks and asking only for the connection section, or requiring patch-style output instead of a whole script.

VIII. QUALITY BEYOND PASS/FAIL

For the 117 passing layouts (8 references, 21 v1, 40 v2, 40 v3) we measured the cell bounding box, the union area of li1, metal1 and metal2 (converted to a length proxy by the minimum width) and contact counts on the flattened layout.

TABLE VIII
THREE CONDITIONS, 8 CELLS $\times$ 5 ATTEMPTS.

condition	API	repair	passed	first-turn passes	fix rounds	tokens in / cached / out	cost
v1	laygo2	human $\leq$ L3	21/40	0/40	225	2.69 M / 0 / 0.36 M	$\approx$ \$19
v2	laygo2_safe	human $\leq$ L3	40/40	13/40	41	0.76 M / 0.58 M / 0.08 M	$\approx$ \$2
v3	laygo2_safe (+fixes)	automatic, L1	40/40	24/40	56	0.95 M / 0.75 M / 0.10 M	$\approx$ \$2.4

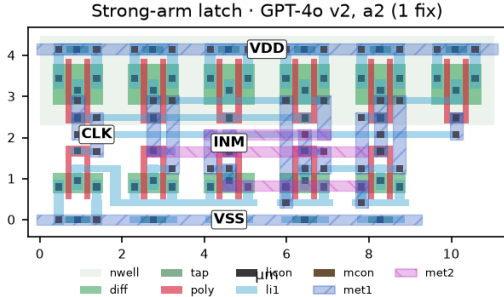


Fig. 6. A strong-arm latch produced by the model on the layer, one instruction after the first turns (v2). The baseline model never got this cell past the crash stage.

Area equals the reference or is smaller (mean ratio 0.98; MUX 16 % smaller). li1 + metal1 length is within $\pm 5\%$ for the small cells. On the larger cells the automatic router uses more metal2 than the human designer (strong-arm latch $26\mu\text{m}$ reference, $42\mu\text{m}$ v2, $61\mu\text{m}$ v3; Fig. 7): when the local-interconnect search fails it escapes upward, where a designer found a lower-layer route. This is the measurable gap between a correct layout and a good one, and the comparison point for the next iteration.

IX. THREATS TO VALIDITY

Confounded changes. Between v1 and v2 the model (05-13 $\rightarrow$ 08-06, for prompt caching), the library and the prompt wording changed together; between v2 and v3 the library and prompts changed as well as the repair mechanism. The clearest evidence for the library effect is qualitative: during development, the same LLM code went from fail to pass after a library fix alone (NOR, TSPC, strong-arm latch). A control run (same library, human instructions) would separate the feedback effect. **Instructor.** Instructions came from coding-agent instances, not an expert designer; help differed between instructors. A pilot run that used L4 instructions was excluded from the baseline. **Comparison with the paper.** The paper does not state how

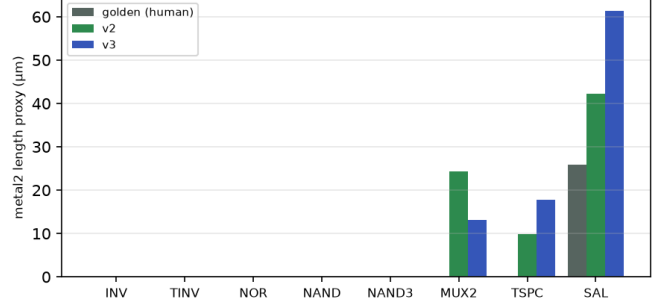


Fig. 7. Metal2 length proxy per cell: human reference versus the layer under human instructions (v2) and automatic feedback (v3).

prompts were counted, the process, success rates or manual edits (it mentions manual adjustments for the level shifter). Our prompt totals (six context turns plus instructions) are not directly comparable. **Scope.** Eight transistor-level cells with at most eleven transistors, two fingers per device; the high-speed cells are connectivity references only; gate-level cells (latch, D flip-flop) have reference layouts but were not benchmarked; five attempts per condition. **Tooling.** The judge, the layer and the loops were implemented with a coding agent; all design decisions are logged with reasons in the repository.

X. RELATED WORK

Generator-based layout frameworks [1], [2], [3] provide the programmable substrate; rule- and template-based analog layout automation (e.g. ALSYN, ALIGN) targets the same cells by other means. LLMs for chip design have mostly produced HDL; You et al. [4] were among the first to generate layout generator code interactively. Our contribution is orthogonal to the model: a measurement protocol and an interface layer that turn the model’s failures into messages a loop can act on.

XI. CONCLUSION

Reproduced on an open-source stack, the paper’s interactive LLM layout flow passes 21 of 40 attempts on eight small cells, and none without human help. Labelling every failure shows that the bottleneck is the interface between the LLM and the framework, not circuit knowledge. A thin net-aware layer that validates inputs, tracks occupancy and finds free tracks removes that bottleneck: 40 of 40 with human help, and 40 of 40 with no human when the layer’s messages are fed back verbatim. The automatic loop’s remaining weakness is oscillation between two failure states on the largest cells, which

neither a problem history nor a script diff cured. The remaining gaps are quality (more metal2 than a designer would use) and scale (larger and gate-level cells), both now measurable.

REPRODUCIBILITY

Pinned submodules, the judge, prompts, the protocol, instructor briefs, per-attempt logs (prompts, replies, generated code, DRC/LVS outputs, labels) and CSV summaries are at <https://github.com/Toragonite/laygo2-llm-agent>. Runs are tagged 20260930 (v1), v2_20260930 (v2), v3_selfheal (v3), v3b_selfheal (problem-history variant) and v4_selfheal (diff variant).

APPENDIX A PER-CELL NUMBERS

TABLE IX
PER CELL: V1 PASSED / MEAN INSTRUCTIONS WHEN PASSED; V2 AND V3
FIRST-TURN PASSES / PASSED / MEAN FIX ROUNDS OVER ALL FIVE
ATTEMPTS.

cell	v1	v2	v3
INV	5/5 · 2.2	5 · 5/5 · 0.0	5 · 5/5 · 0.0
TINV	5/5 · 2.4	5 · 5/5 · 0.0	5 · 5/5 · 0.0
NOR2	5/5 · 4.6	2 · 5/5 · 0.8	4 · 5/5 · 0.4
NAND2	1/5 · 7.0	0 · 5/5 · 1.2	4 · 5/5 · 0.4
NAND3	5/5 · 4.0	1 · 5/5 · 0.8	4 · 5/5 · 0.4
2-to-1 MUX	0/5	0 · 5/5 · 1.0	0 · 5/5 · 3.4
TSPC FF	0/5	0 · 5/5 · 1.6	2 · 5/5 · 1.4
strong-arm latch	0/5	0 · 5/5 · 2.8	0 · 5/5 · 5.2

REFERENCES

- [1] E. Chang et al., “BAG2: A process-portable framework for generator-based AMS circuit design,” in *Proc. IEEE CICC*, 2018.
- [2] J. Han, W. Bae, E. Chang, Z. Wang, B. Nikolić, and E. Alon, “LAYGO: A template-and-grid-based layout generation engine for advanced CMOS technologies,” *IEEE Trans. Circuits Syst. I*, vol. 68, no. 3, pp. 1012–1022, 2021.
- [3] T. Shin et al., “LAYGO2: A custom layout generation engine based on dynamic templates and grids for advanced CMOS technologies,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 12, pp. 4402–4412, 2023.
- [4] G. You, Y. Byun, S. Lim, and J. Han, “Interactive and automatic generation of primitive custom circuit layout using LLMs,” *MLCAD 2024 WIP*, arXiv:2408.07279, 2024.